

Lab 06: Anti-Forensics & Cryptography

F20FO/F21FO – Digital Forensics

Heriot-Watt University

2025–26

Exam-Ready Study Notes

Beginner-Friendly • Step-by-Step • Zero Prior Knowledge Assumed

Topics Covered:

Symmetric Encryption • Caesar & Shift Ciphers • Substitution Ciphers
Frequency Analysis • Vigenère Cipher • Asymmetric Encryption
GPG/Public Key Cryptography • Null Ciphers & Steganography
Word Shifting • Password Hashing • Password Cracking
John the Ripper • Brute Force vs Dictionary Attacks

Contents

1	The Big Picture: What Are We Learning?	2
2	Symmetric Key Encryption	3
2.1	The Shift Cipher (Caesar Cipher)	3
2.1.1	How It Works – Step by Step	3
2.2	General Substitution Cipher	4
2.3	Frequency Analysis – How to Crack Substitution Ciphers	5
2.3.1	Step-by-Step: How to Do Frequency Analysis	5
2.4	The Vigenère Cipher	6
2.4.1	How It Works – Step by Step	6
2.4.2	How to Decrypt the Vigenère Cipher	7
3	Asymmetric Encryption (Public Key Cryptography)	8
3.1	GPG (GNU Privacy Guard)	8
3.1.1	Digital Signatures vs Encryption	9
4	Steganography (Hiding Messages in Plain Sight)	10
4.1	Null Cipher	10
4.2	Word Shifting	10
5	Password Storage and Cracking	12
5.1	How Passwords Are Stored	12
5.1.1	Where Are Password Hashes Stored?	12
5.2	Password Cracking Methods	13
5.3	John the Ripper	13
5.3.1	Key Commands	13
5.3.2	The Password File Format	14
5.4	Common Hashing Algorithms	14
6	Python Coding for Forensics	15
6.1	Reverse Cipher (Python)	15
6.2	Caesar Cipher (Python)	15
7	Complete Summary & Cheat Sheet	16
8	Practice Questions for Exam Preparation	17

1 The Big Picture: What Are We Learning?

What is Anti-Forensics?

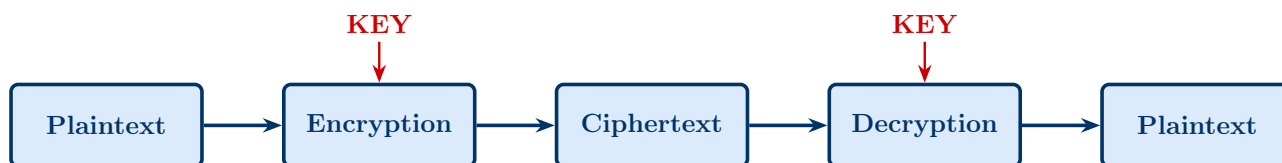
Anti-forensics is anything someone does to **hide, destroy, or obscure digital evidence** so that forensic investigators can't find or read it. Think of it like a criminal wiping fingerprints off a door handle – but in the digital world.

Cryptography is one of the biggest anti-forensics tools: it turns readable data into unreadable gibberish unless you have the right key.

What is Cryptography?

Cryptography = the science of **secret writing**. It's how we protect data by scrambling it so only authorised people can read it.

- **Plaintext** = the original, readable message (e.g., "HELLO")
- **Ciphertext** = the scrambled, unreadable message (e.g., "KHOOR")
- **Encryption** = the process of turning plaintext → ciphertext
- **Decryption** = the process of turning ciphertext → plaintext
- **Key** = the secret ingredient that controls the encryption/decryption



Exam Tip: The Two Types of Encryption

There are only **two main types** of encryption you need to know. Everything in this lab falls into one of these:

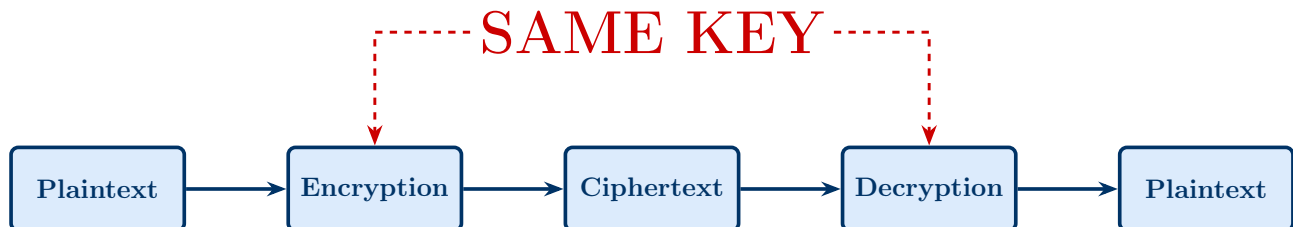
Feature	Symmetric	Asymmetric
Keys used	ONE key (same for encrypt & decrypt)	TWO keys (public + private)
Analogy	A padlock where everyone shares the same key	A mailbox: anyone can put mail in (public key), only you can open it (private key)
Main problem	How do you safely share the key?	Slower; more computationally expensive
Examples	Caesar cipher, Vigenère, AES, DES	RSA, GPG/PGP

2 Symmetric Key Encryption

Definition: Symmetric Key Encryption

Symmetric encryption uses **the same key** for both encrypting and decrypting data. Think of it like a locked diary: you use the same key to lock it (encrypt) and unlock it (decrypt).

The big weakness: You **MUST** share the key with anyone you want to communicate with. If the key gets intercepted, the entire encryption is broken.



2.1 The Shift Cipher (Caesar Cipher)

Definition: Shift Cipher

A shift cipher replaces each letter with another letter a **fixed number of positions** down the alphabet. The number of positions is the **key**.

Caesar Cipher = a shift cipher with a key of **3**.

It is named after Julius Caesar, who used it to communicate with his generals.

2.1.1 How It Works – Step by Step

Example: Encrypting “HELLO” with a Caesar Cipher (shift = 3)

Step 1: Write out the alphabet and number each letter 0–25:

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

Step 2: For each letter, move forward by 3 positions:

Plaintext Letter	Position	Position + 3	Ciphertext Letter
H	7	$7 + 3 = 10$	K
E	4	$4 + 3 = 7$	H
L	11	$11 + 3 = 14$	O
L	11	$11 + 3 = 14$	O
O	14	$14 + 3 = 17$	R

Result: HELLO → **KHOOR**

To decrypt: Move each letter **backwards** by 3 positions. K→H, H→E, O→L, O→L, R→O = HELLO

What happens when we go past Z? – The Wrap-Around Rule

If shifting pushes a letter past Z, you **wrap around** to the beginning of the alphabet.

Example: Encrypt “XYZ” with shift = 3

- X (position 23) + 3 = 26 → $26 \bmod 26 = 0$ → **A**

- Y (position 24) + 3 = 27 $\rightarrow$ 27 mod 26 = 1 $\rightarrow$ **B**
- Z (position 25) + 3 = 28 $\rightarrow$ 28 mod 26 = 2 $\rightarrow$ **C**

The formula: Ciphertext position = (Plaintext position + Key) mod 26

What is “mod”? It means “remainder after dividing.” So $28 \div 26 = 1$ remainder 2, therefore $28 \bmod 26 = 2$.

Lab Exercise: “CAESAR” with a shift of 6

Letter	Position	+6	Result
C	2	2+6=8	I
A	0	0+6=6	G
E	4	4+6=10	K
S	18	18+6=24	Y
A	0	0+6=6	G
R	17	17+6=23	X

Answer: CAESAR with shift 6 = **IGKYGX**

Lab Exercise: What key makes “CAESAR” become “MKOCKB”?

Compare first letters: C $\rightarrow$ M. C is position 2, M is position 12.

$12 - 2 = 10$, so the key is **10**.

Verify: A(0) $\rightarrow$ K(10) $\checkmark$, E(4) $\rightarrow$ O(14) $\checkmark$, S(18) $\rightarrow$ C(2, since $28 \bmod 26=2$) $\checkmark$, A(0) $\rightarrow$ K(10) $\checkmark$, R(17) $\rightarrow$ B(1, since $27 \bmod 26=1$) $\checkmark$

Answer: Key = **10**

Lab Exercise: Decrypt “Kbtp fp fkcljxqflk xylrq zroobkq bsbkqp”

Let’s try shifting backwards by different amounts. Look at small common words for clues.

“fp” is a 2-letter word $\rightarrow$ probably “is”, “it”, “an”, “at”, etc.

If f $\rightarrow$ i, then shift = 3. Let’s check: p $\rightarrow$ s (p=15, $15+3=18=s$). “fp” $\rightarrow$ “is” $\checkmark$

Decrypt the whole thing with shift = 3 backwards (or shift = 23 forward):

Plaintext: “**This is information about current events.**”

Key = 23 (or equivalently, a backward shift of 3)

Exam Tip: Why Is the Shift Cipher Weak?

- There are only **25 possible keys** (shifts 1–25; shift 0 means no encryption)
- An attacker can simply try all 25 possibilities – this is called a **brute-force attack**
- It takes seconds to crack, even by hand

2.2 General Substitution Cipher

Definition: Substitution Cipher

A more general version of the shift cipher where each letter is mapped to a **different specific letter**, but the mapping is not necessarily a simple shift. Each letter has its own unique replacement.

Lab Exercise: Custom Substitution Table

Given this mapping table from the lab:

Plain	C	D	E	H	I	N	P	R	S	T	Y
Cipher	X	J	L	A	Z	E	V	K	H	O	M

Ciphertext: JLXKMVO OAZH

To decrypt, go **backwards** through the table (cipher → plain):

Cipher	J	L	X	K	M	V	O		O	A	Z	H
Plain	D	E	C	R	Y	P	T		T	H	I	S

Answer: JLXKMVO OAZH = **DECRYPT THIS**

2.3 Frequency Analysis – How to Crack Substitution Ciphers

Definition: Frequency Analysis

Frequency analysis exploits the fact that **letters in English are not used equally**. The letter ‘e’ appears about 12.7% of the time, while ‘z’ appears only 0.1%. By counting how often each letter appears in the ciphertext, you can make educated guesses about which cipher letter = which plain letter.

Exam Tip: Memorise the Top 5 Most Common English Letters

E – T – A – O – I – N – S – H – R

Letter	e	t	a	o	i	n	s	h	r	d	l	u
Frequency %	12.7	9.1	8.2	7.5	7.0	6.7	6.3	6.1	6.0	4.3	4.0	2.8

Memory trick: “ETAOIN SHRDLU” – these are the 12 most common letters in order. The most frequent letter in your ciphertext is probably ‘E’.

2.3.1 Step-by-Step: How to Do Frequency Analysis

1. **Count every letter** in the ciphertext and record the frequency
2. **Rank them** from most to least frequent
3. **Map the most frequent ciphertext letter to ‘E’** (the most common English letter)
4. **Map the second most frequent to ‘T’**, third to ‘A’, etc.
5. **Look for patterns:** common 1-letter words (I, A), common 2-letter words (IS, IT, IN, AT), common 3-letter words (THE, AND, FOR)
6. **Refine by trial and error** – adjust mappings until the plaintext makes sense

Important: Frequency Analysis Only Works Well on Long Texts

The longer the ciphertext, the more accurate the frequency counts. Short messages may not follow normal letter frequencies, making analysis harder.

2.4 The Vigenère Cipher

Definition: Vigenère Cipher

The Vigenère cipher is a **polyalphabetic** cipher – this fancy word just means it uses **multiple different shifts** instead of one. Each letter in your message gets shifted by a **different amount**, determined by a keyword (passphrase).

Invented by: Blaise de Vigenère (16th century French cryptographer)

Why it's better than Caesar: Because each letter is shifted by a different amount, frequency analysis is MUCH harder. The letter 'E' won't always become the same cipher letter.

2.4.1 How It Works – Step by Step

Example: Encrypting “VIGENERE CIPHER” with keyword “COUNTON”

Step 1: Write the keyword above the plaintext, repeating it as needed:

Key	C	O	U	N	T	O	N	C	O	U	N	T	O	N
Plaintext	V	I	G	E	N	E	R	E	C	I	P	H	E	R

Step 2: For each pair, the key letter tells you **which row** to use, and the plaintext letter tells you **which column** to use in the Vigenère table. The intersection is the ciphertext.

Simple way to think about it: Each key letter represents a shift amount (A=0, B=1, C=2, ..., Z=25).

Key Letter	Shift Amount	Plaintext	Calculation	Cipher
C	2	V (21)	$(21+2) \bmod 26 = 23$	X
O	14	I (8)	$(8+14) \bmod 26 = 22$	W
U	20	G (6)	$(6+20) \bmod 26 = 0$	A
N	13	E (4)	$(4+13) \bmod 26 = 17$	R
T	19	N (13)	$(13+19) \bmod 26 = 6$	G
O	14	E (4)	$(4+14) \bmod 26 = 18$	S
N	13	R (17)	$(17+13) \bmod 26 = 4$	E
C	2	E (4)	$(4+2) \bmod 26 = 6$	G
O	14	C (2)	$(2+14) \bmod 26 = 16$	Q
U	20	I (8)	$(8+20) \bmod 26 = 2$	C
N	13	P (15)	$(15+13) \bmod 26 = 2$	C
T	19	H (7)	$(7+19) \bmod 26 = 0$	A
O	14	E (4)	$(4+14) \bmod 26 = 18$	S
N	13	R (17)	$(17+13) \bmod 26 = 4$	E

Result: VIGENERE CIPHER → **XWARGSEGQCCASE**

2.4.2 How to Decrypt the Vigenère Cipher

Decryption: Reverse the process

To decrypt, **subtract** the key shift instead of adding:

Formula: Plaintext = (Ciphertext position – Key position) mod 26

If the result is negative, add 26.

Example: Ciphertext = X (23), Key = C (2): $(23 - 2) \bmod 26 = 21 = V$ ✓

Lab Exercise: Decrypt “Mjxa xl Xxotgggm Rbrwmg” with key TCPIP

Step 1: Write the key above the ciphertext (skipping spaces):

Key	T	C	P	I		P	I		T	C	P	I	P	I	P	I		T	C	P	I	P	I
Cipher	M	j	x	a		x	l		X	x	o	t	g	g	g	m		R	b	r	w	m	g

Step 2: Subtract key from cipher for each letter:

$M(12) - T(19) = -7 \bmod 26 = 19 = \mathbf{T}$

$j(9) - C(2) = 7 = \mathbf{H}$

$x(23) - P(15) = 8 = \mathbf{I}$

$a(0) - I(8) = -8 \bmod 26 = 18 = \mathbf{S}$

Continuing this process...

Answer: “This is Edinburgh Castle”

Exam Tip: Vigenère vs Simple Substitution

Why is Vigenère more secure?

- In a simple substitution, the letter ‘E’ *always* becomes the same cipher letter, so frequency analysis works easily
- In Vigenère, the letter ‘E’ becomes *different* cipher letters depending on its position, because the key letter changes each time
- This **flattens the frequency distribution**, making it much harder to crack
- However, if you know the key length, you can still use frequency analysis on each “column” separately

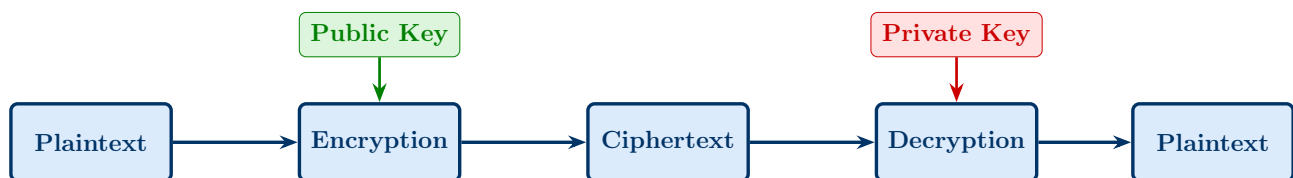
3 Asymmetric Encryption (Public Key Cryptography)

Definition: Asymmetric Encryption

Asymmetric encryption uses **TWO different keys**:

- **Public Key** – given to everyone openly; used to **encrypt** data
- **Private Key** – kept secret by the owner; used to **decrypt** data

Real-world analogy: Think of a **mailbox**. Anyone can drop a letter in through the slot (encrypt with public key), but only the person with the mailbox key can open it and read the letters (decrypt with private key).



Exam Tip: Symmetric vs Asymmetric – The Key Difference

Aspect	Symmetric	Asymmetric
Number of keys	1 (shared)	2 (public + private)
Key distribution	MAJOR PROBLEM: must share key securely	SOLVED: public key is openly shared
Speed	Fast	Slower (complex maths)
Use case	Encrypting large amounts of data	Secure key exchange, digital signatures
If key is stolen	Everything is compromised	Only private key matters; public key is already public

3.1 GPG (GNU Privacy Guard)

What is GPG?

GPG (GNU Privacy Guard) is a free, real-world tool that implements **asymmetric encryption**. It lets you:

- **Generate** a public/private key pair
- **Encrypt** messages using someone's public key
- **Decrypt** messages using your private key
- **Digitally sign** messages to prove they came from you
- **Verify** that a signed message hasn't been tampered with

GPG4Win is the Windows version, which includes:

- **GnuPG** – the core encryption engine
- **Kleopatra** – key management tool (manage your certificates/keys)
- **GPA** – GNU Privacy Assistant (clipboard-based encrypt/sign/verify)

3.1.1 Digital Signatures vs Encryption

Feature	Digital Signature	Encryption
Purpose	Proves WHO sent the message (authentication)	Hides the MESSAGE content (confidentiality)
Who can read it?	Everyone (message is not hidden)	Only the recipient with the private key
Uses which key?	Sender signs with THEIR private key ; receiver verifies with sender's public key	Sender encrypts with RECIPIENT's public key ; recipient decrypts with THEIR private key
When to use	Verifying authenticity of legal documents, software updates, forensic reports	Sending confidential data: medical records, financial info, case evidence

Exam Tip: GPG Key Concepts

- The **passphrase** protects your private key (like a password for your key)
- The security assumption is that **only you have access to your private key**
- GPG enables: **secure communication**, **data encryption**, and **sender authentication**
- All of these are critical for **forensic investigators** and organisations protecting sensitive data

4 Steganography (Hiding Messages in Plain Sight)

Definition: Steganography

Steganography is the art of **hiding a secret message inside something that looks completely normal**. Unlike encryption (which makes data unreadable), steganography makes the data **invisible** – the attacker doesn't even know a secret message exists.

Key difference from encryption:

- **Encryption** = everyone can see there IS a secret message, they just can't read it
- **Steganography** = nobody even knows a secret message EXISTS

4.1 Null Cipher

Definition: Null Cipher

A null cipher hides a message within an innocent-looking message. You need to know the “key” (the pattern) to extract the hidden message. Common patterns include taking the first letter of each word, the last letter, or a specific position.

Example 1: First Letter of Each Word

“News Eight Weather: Tonight increasing snow. Unexpected precipitation smothers eastern towns...”

Taking the first letter of each word: **N-E-W-T-I-S-U-P-S-E-T...**

Hidden message: “**Newt is upset because he thinks he is President.**”

Example 2: Pattern-Based (1st, 2nd, 3rd letter)

“Susan sAys GaIl Lies. MAtt leTs Susan fEel joVial. Elated/aNgry?”

Pattern: take the 1st letter, then 2nd, then 3rd, cycling through.

Hidden message: “**Sail at seven.**”

Lab Exercise: Null Cipher with Fixed Position

“The hairy elephant, came and took salt and tea over near the hazardous energy making apparatus today”

Taking the **first letter** of each word (skipping “The”) or second letter:

If we take the **second letter** of each word: The hairy elephant, came and took salt and tea over near the hazardous energy making apparatus today

Hidden message: “**he can eat one at a time**” – or reading more carefully, “**halanoanevehan-napo**” – Let's try the first letters:

T-h-e-c-a-t-s-a-t-o-n-t-h-e-m-a-t = “**the cat sat on the mat**”

Answer: “the cat sat on the mat” (using first letter of each word)

4.2 Word Shifting

Definition: Word Shifting

Word shifting hides a message by placing **extra spaces before certain words** in a paragraph. The words with an extra space before them, read in order, form the hidden message. You typically need to paste the text into a text editor and carefully look for double spaces.

Lab Example: The Two Messages

Compare Message 1 and Message 2 in the lab. They look identical, but Message 2 has an **extra space** before: *explore, the, world, wide, web*.

Hidden message: “**explore the world wide web**”

Exam Tip: Why Does Steganography Matter for Forensics?

- Criminals can use null ciphers and word shifting to communicate covertly
- The messages look completely innocent – a forensic investigator might miss them
- Steganography can also hide data in **images**, **audio files**, and **video**
- A forensic investigator must be aware of these techniques to look for hidden evidence

5 Password Storage and Cracking

5.1 How Passwords Are Stored

Key Concept: Passwords Are NOT Stored as Plain Text

Modern systems do NOT store your actual password. Instead, they store a **hash** of your password. **What is a hash?** A hash is a one-way mathematical function that takes any input and produces a fixed-length output (a “fingerprint”).

Key properties:

- **One-way:** You can go from password → hash, but you CANNOT go from hash → password
- **Deterministic:** The same password ALWAYS produces the same hash
- **Fixed length:** No matter if the password is 4 characters or 40, the hash is always the same length
- **Small change = big difference:** Changing one letter completely changes the hash

Analogy: Hashing is Like a Meat Grinder

- You put a steak in a meat grinder → you get mince (this is like hashing)
- You CANNOT put the mince back together to get the original steak (one-way)
- The same steak always produces the same mince (deterministic)
- A slightly different steak produces completely different-looking mince (avalanche effect)

5.1.1 Where Are Password Hashes Stored?

Operating System	Hash Location	Hash Type
Windows	system32\config\SAM	LMHash (LANMAN) and NT Hash
Linux	/etc/passwd or /etc/shadow	DES, MD5, SHA-256, SHA-512

Important: What is a Salt?

A **salt** is a random string added to the password **before** hashing.

Why? Without salt, two users with the same password would have the same hash. An attacker could pre-compute a giant table of hashes (called a **rainbow table**) and look up passwords instantly.

With salt:

- Password “1234” + Salt “xK9” → Hash of “1234xK9” → completely different hash
- Password “1234” + Salt “mQ7” → Hash of “1234mQ7” → another completely different hash

Same password, different salts = different hashes. This defeats rainbow tables.

5.2 Password Cracking Methods

Method	How It Works	Speed	Effectiveness
Brute Force	Tries EVERY possible combination (a, b, c, ..., aa, ab, ...)	Very slow for long passwords	Guaranteed to work eventually
Dictionary Attack	Uses a list of common words/-passwords and tries each one	Much faster	Only works if the password is in the dictionary
Rainbow Table	Pre-computed table mapping hashes to passwords	Very fast lookup	Defeated by salting

Analogy: Understanding the Three Methods

Imagine you're trying to open a 4-digit combination lock:

Brute Force: Try 0000, 0001, 0002, ..., 9999. You WILL find it, but it takes up to 10,000 tries.

Dictionary Attack: You know the person likes using birthdays, so you only try combinations that look like dates (0101, 1225, 0704, etc.). Much fewer tries, but only works if the combo IS a date.

Rainbow Table: Someone already tried every combination and wrote down "lock shape XYZ = combo 4832". You just look up your lock shape in the book. Instant – but the book is huge, and doesn't work if the lock has been modified ("salted").

5.3 John the Ripper

What is John the Ripper?

John the Ripper is a popular **open-source password cracking tool**. It can crack passwords hashed with DES, MD5, SHA, NT Hash, and many other formats.

How it works: It takes a file containing username:hash pairs and tries to figure out the original passwords.

5.3.1 Key Commands

Command	What It Does
<code>john pass.txt</code>	Default brute-force crack on the hash file
<code>john -i:alpha pass.txt</code>	Brute force using only letters (a-z)
<code>john -i:digits pass.txt</code>	Brute force using only numbers (0-9)
<code>john -i:all pass.txt</code>	Brute force using all characters (letters + numbers + symbols)
<code>john -w:word_list.txt pass.txt</code>	Dictionary attack using a custom word list
<code>john -format=raw-md5 pass.txt</code>	Crack an MD5 hash (specify the hash format)
<code>john -show pass.txt</code>	Show previously cracked passwords

Important Note About John the Ripper

- Once John cracks a password, it saves the result in `john.pot`
- If you want to re-crack the same hash file, you must **delete** `john.pot` from the `run` directory first
- John does NOT show progress – you just have to wait
- Simple passwords (like “1234”) crack in seconds; complex ones can take hours, days, or longer

5.3.2 The Password File Format

How the hash file looks

John expects a text file where each line has the format:

`username:hash`

Example: `u1:CRP0W3oTu3Z4s`

This means user “u1” has the DES hash “CRP0W3oTu3Z4s”. John will try to figure out what password produces this hash.

5.4 Common Hashing Algorithms

Algorithm	Output Length	Security	Notes
DES	13 characters	Weak (easily cracked)	Old standard; John cracks it very well
MD5	32 hex characters	Weak (collision-prone)	Still widely used but insecure
SHA-1	40 hex characters	Weak	Deprecated; replaced by SHA-2
SHA-256	64 hex characters	Strong	Part of SHA-2 family; widely used
LMHash	32 hex characters	Very weak	Windows legacy; splits password into 7-char chunks
NT Hash	32 hex characters	Moderate	Modern Windows; MD4-based

Exam Tip: Known Flaws in Windows Hashing

- **LMHash (LANMAN)** is extremely weak because it converts passwords to uppercase and splits them into two 7-character halves, each hashed separately. This means cracking a 14-character password is only as hard as cracking two 7-character passwords!
- **NT Hash** is better but still has no salting by default, making it vulnerable to rainbow table attacks
- Both LMHash and NT Hash have “known flaws”

6 Python Coding for Forensics

Why Python Matters in Forensics

Python allows forensic investigators to:

- **Automate** repetitive tasks (like trying all 25 Caesar cipher keys)
- **Build custom tools** for analysis
- **Verify results** – if you decrypt something by hand, use Python to double-check

6.1 Reverse Cipher (Python)

What the reverseCipher.py Does

This script simply prints a string **backwards**.

Input: “Hello and welcome to DF” → Output: “FD ot emoclew dna olleH”

Does this offer security? No! Reversing a string is trivially easy to undo. Anyone who sees the message can immediately reverse it back. It’s an example of **security through obscurity**, which is no real security at all.

6.2 Caesar Cipher (Python)

How caesarCipher.py Works

The script takes a message and a shift key, then applies the shift to each letter. In the lab, you modify it to:

1. Ask the user whether they want to **encrypt** or **decrypt**
2. Ask which **key (shift number)** to use

For encryption: shift each letter **forward** by the key amount.

For decryption: shift each letter **backward** by the key amount (or forward by $26 - \text{key}$).

7 Complete Summary & Cheat Sheet

Master Comparison Table: All Encryption/Hiding Methods

Method	Type	Key	Security	Attack	Key Weakness
Caesar/Shift Cipher	Symmetric	Single number (1–25)	Very weak	Brute force (25 tries)	Only 25 possible keys
Substitution Cipher	Symmetric	Letter mapping table	Weak	Frequency analysis	Letter frequencies preserved
Vigenère Cipher	Symmetric	Keyword (passphrase)	Moderate	Frequency analysis per column	Repeating key pattern
GPG/RSA	Asymmetric	Public + Private key pair	Strong	Factoring large primes	Key must be long enough
Null Cipher	Steganography	Position pattern	Weak if pattern known	Pattern recognition	Only obscurity, not encryption
Word Shifting	Steganography	Extra spaces	Very weak	Text editor inspection	Easy to spot digitally

Password Cracking Cheat Sheet

Concept	Definition	Example	Key Point
Hash	One-way function; password → fixed-length output	“1234” → CRP0W3oTu3Z4s	Cannot reverse a hash
Salt	Random string added before hashing	“1234” + “xK9” → hash	Defeats rainbow tables
Brute Force	Try every possible combination	a, b, ..., aa, ab, ...	Slow but guaranteed
Dictionary Attack	Try words from a list	common passwords, names	Fast but limited
Rainbow Table	Pre-computed hash lookup	Hash → password in table	Defeated by salting
SAM file	Windows hash storage	system32\config\SAM	Contains user hashes
/etc/shadow	Linux hash storage	/etc/shadow	More secure than /etc/passwd

8 Practice Questions for Exam Preparation

Practice Question 1: Caesar Cipher

Q: Encrypt the word “FORENSICS” using a Caesar cipher with a key of 5.

A: $F(5)+5=10=K$, $O(14)+5=19=T$, $R(17)+5=22=W$, $E(4)+5=9=J$, $N(13)+5=18=S$, $S(18)+5=23=X$, $I(8)+5=13=N$, $C(2)+5=7=H$, $S(18)+5=23=X$

Answer: FORENSICS → KTWJSXNHX

Practice Question 2: Symmetric vs Asymmetric

Q: Explain why asymmetric encryption solves the key distribution problem of symmetric encryption.

A: In symmetric encryption, both sender and receiver need the same secret key, which must be shared over a potentially insecure channel – if intercepted, all communication is compromised. In asymmetric encryption, the public key can be freely shared with anyone (even posted online) because knowing the public key does NOT allow you to decrypt messages. Only the private key, which never leaves the owner’s possession, can decrypt. This eliminates the need to securely transmit a shared secret.

Practice Question 3: Hashing and Salting

Q: Why is salting important for password security? What attack does it prevent?

A: Without salt, two users with the same password produce the same hash. This means an attacker can build a rainbow table (a pre-computed database of hash-to-password mappings) and look up passwords instantly. Salting adds a unique random string to each password before hashing, so even identical passwords produce different hashes. This forces the attacker to crack each hash individually, making rainbow tables useless.

Practice Question 4: Frequency Analysis

Q: Why does frequency analysis work on simple substitution ciphers but NOT on the Vigenère cipher?

A: In a simple substitution cipher, each plaintext letter always maps to the same ciphertext letter (e.g., E always becomes P). This preserves the natural frequency distribution of English letters, so the most common ciphertext letter is likely ‘E’. In the Vigenère cipher, the same plaintext letter can become different ciphertext letters depending on its position (because the key letter changes), which disrupts the frequency pattern and makes simple frequency analysis ineffective.

Practice Question 5: John the Ripper

Q: What is the difference between `john pass.txt` and `john -w:wordlist.txt pass.txt`?

A: The first command performs a **brute-force attack**, trying every possible character combination systematically. The second command performs a **dictionary attack**, trying only words from the specified word list file. The brute-force method is guaranteed to eventually find the password but is much slower. The dictionary attack is faster but only works if the password happens to be in the word list.

Practice Question 6: Digital Signatures

Q: In what circumstances might a forensic investigator use (a) a digital signature and (b) full email encryption?

A: (a) A digital signature would be used when submitting forensic reports to court or to a client, to prove the report is authentic and has not been tampered with since the investigator signed it. It provides non-repudiation (the investigator cannot deny they wrote it).

(b) Full encryption would be used when sending sensitive case evidence, suspect data, or confidential findings to another party. This ensures that even if the email is intercepted, the contents cannot be read by unauthorised parties.

Practice Question 7: Steganography

Q: Why is steganography potentially more dangerous than encryption from a forensic perspective?

A: With encryption, an investigator can at least *see* that encrypted data exists (even if they can't read it) – this alerts them that someone is hiding something. With steganography, the hidden message is invisible within an innocent-looking document, image, or email. The investigator might not even realise there IS hidden data to look for. This makes steganographic evidence much harder to detect during a forensic examination.